



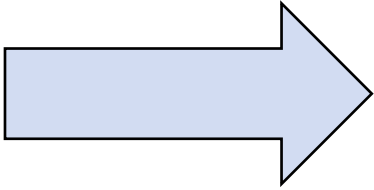
Collaborative Git

INTERSECT Bootcamp

Lauren Milechin



Outline



- Introduction
- Branches
- Pull Requests
- Git Workflows



Challenges of Collaborative Development

- Sharing code such that
 - Everyone has the latest version
 - Everyone can contribute equally
- Keeping organized within the team
 - Everyone is on the same page
 - Communication

Challenges of Collaborative Development

- Sharing code such that
 - Everyone has the latest version
 - Everyone can contribute equally
- Keeping organized within the team
 - Everyone is on the same page
 - Communication





AI Coding Agents and Git

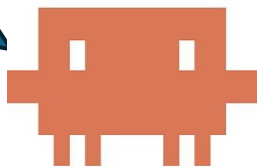
- Agents work better when your code is version controlled with git
 - Access to more context through git history and file diffs (how the file has changed)
 - Testing is more efficient: it can compare results between current and previous versions
- Agents can help with git, but still need oversight
 - Can run git commands, write commit messages, make and review pull requests
 - Won't follow conventions or best practices unless you instruct it to
 - Agent config files (AGENTS.md, CLAUDE.md, copilot-instructions.md) help, but not perfect



AI Coding Agents and Git

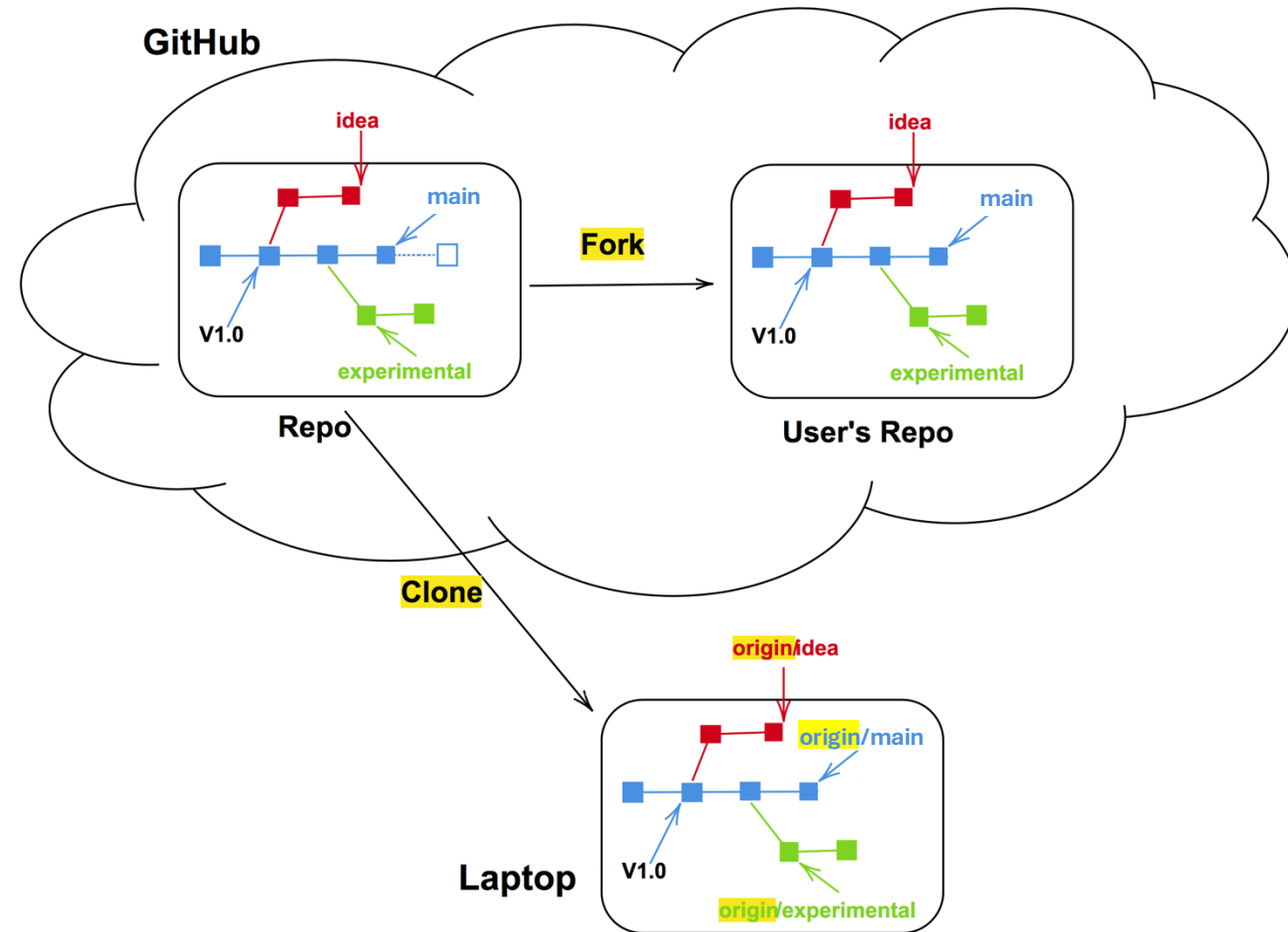
The advice is to add a CLAUDE.md file to repositories so agents know how you like to work in the repo. But every time I try to test it with a new feature Claude ignores it and instead starts working. Is there something I am doing wrong?

Nothing wrong on your end — this is a known gap. CLAUDE.md gets read as background context during exploration (it's just another file among several I read to understand the codebase), so its prose gets absorbed as "useful info" rather than treated as a mandatory gate that has to fire before the first edit. Imperative instructions buried in markdown compete with everything else in context, and "read the rules" and "act on the rules" aren't the same event to me unless something forces the connection — which is exactly what happened here: I read the workflow section, then went straight to editing files without treating it as a checklist to execute.



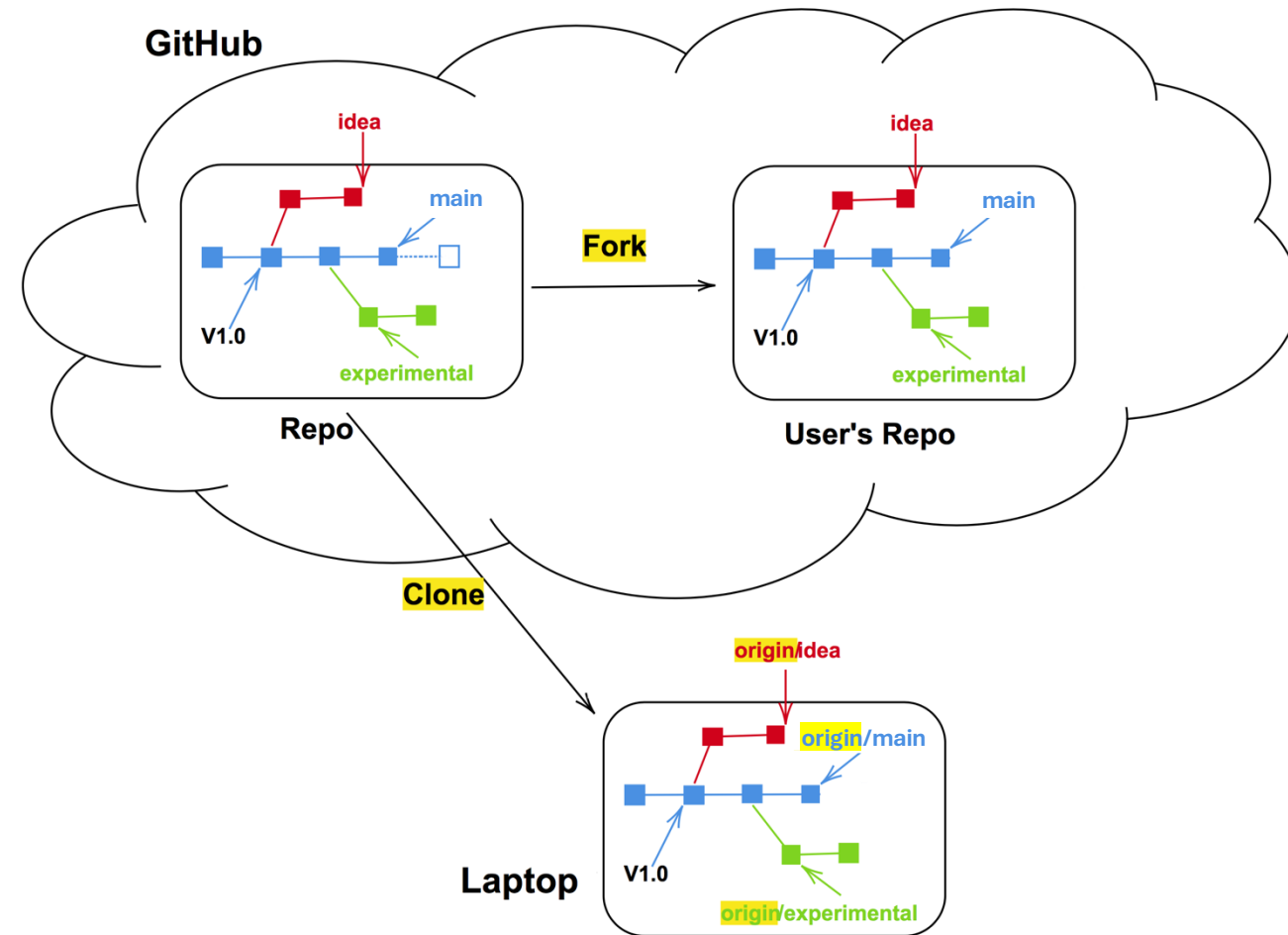
Review- Terminology

- Repository (repo): The project, contains all data and history (commits, branches, tags).
 - Remote/central (repository): The copy of the repository on a remote server, such as GitHub. Accessible to anyone who has access to the repository on the server.
 - Local (repository): A copy of the repository in another place, such as a laptop or your home directory on a cluster. Generally accessible to one person.
- Commit: Snapshot of the project, gets a unique identifier (e.g. c7f0e8bfc718be04525847fc7ac237f470add76e).
- Tag: A pointer to one commit, to be able to refer to it later. Like a commemorative plaque that you attach to a particular commit (e.g. phd-printed or paper-submitted).



Review- Terminology

- Branch: Independent development line. The main development line is often called main or master.
- Cloning: Copying the whole repository to your laptop - the first time. This creates a local copy of the repository.
- Forking: Taking a copy of a repository (which is typically not yours) into your GitHub/GitLab account - your copy (fork) stays on GitHub/GitLab and you can make changes to your copy.
- Pull: Bring changes from a remote repository to your local repository.
- Push: Bring changes from your local repository to the remote repository.



Review- Basic Git Pattern

1. Clone the repository (run once)

```
git clone URL
```

2. Pull any new changes on the remote

```
git pull
```

3. Make changes

4. Commit your changes

```
git add filename
```

```
git commit -m "short descriptive commit message"
```

5. Push your changes to the remote repository

```
git push
```

6. Repeat steps 3-5 as needed.

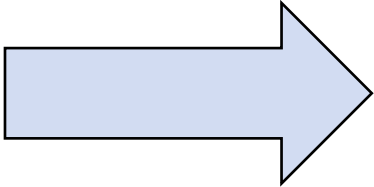


Activity (10 mins)

- We will work in small groups for this module
- Follow along
- One person in each group
 - Create a repository from the template
 - <https://github.com/INTERSECT-training/intersect-training-practice>
 - Use this Template -> Create a New Repository
 - Give access to each member in your group
- Everyone
 - Accept invite to collaborate
 - Clone the repository to your computer with `git clone`



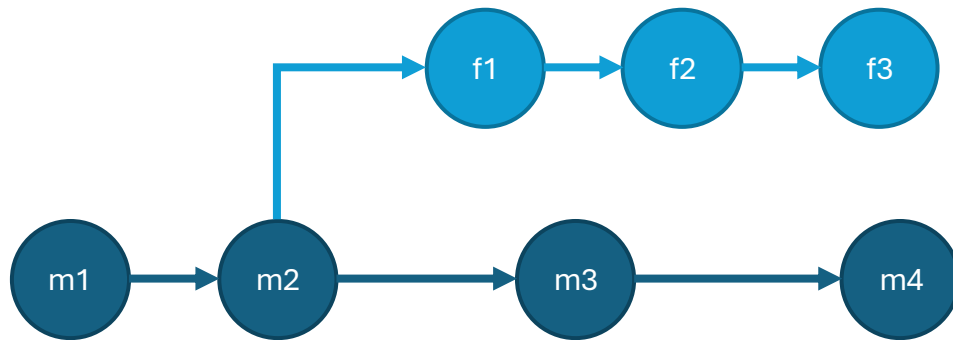
Outline



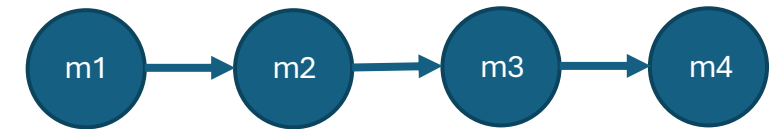
- Introduction
- Branches
- Pull Requests
- Git Workflows

Branches

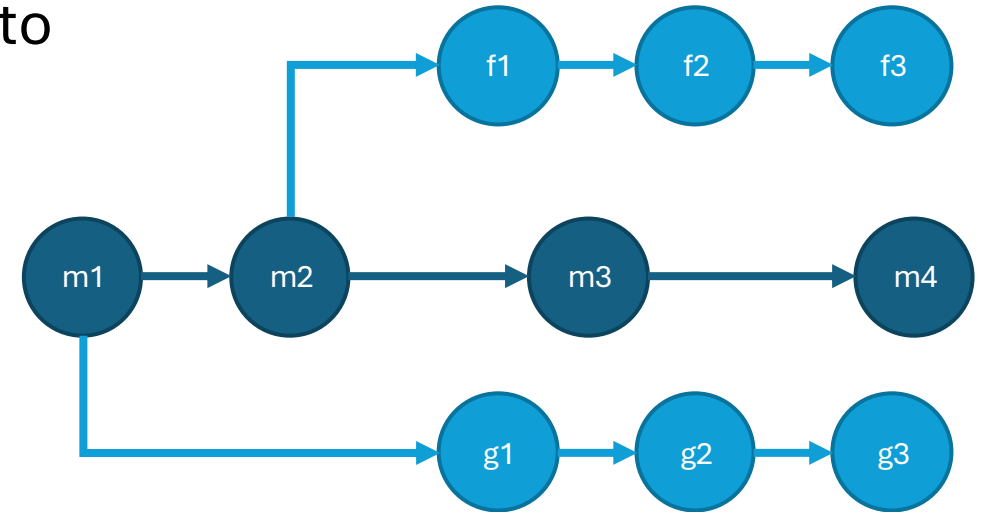
- Independent development lines
- Different purposes (feature, development, production)
- Varying lifetimes (short vs long-lived)
- Commits from one branch can be merged into another



Main Branch with Feature Branch



Main Branch Only



Main Branch with 2 Feature Branches

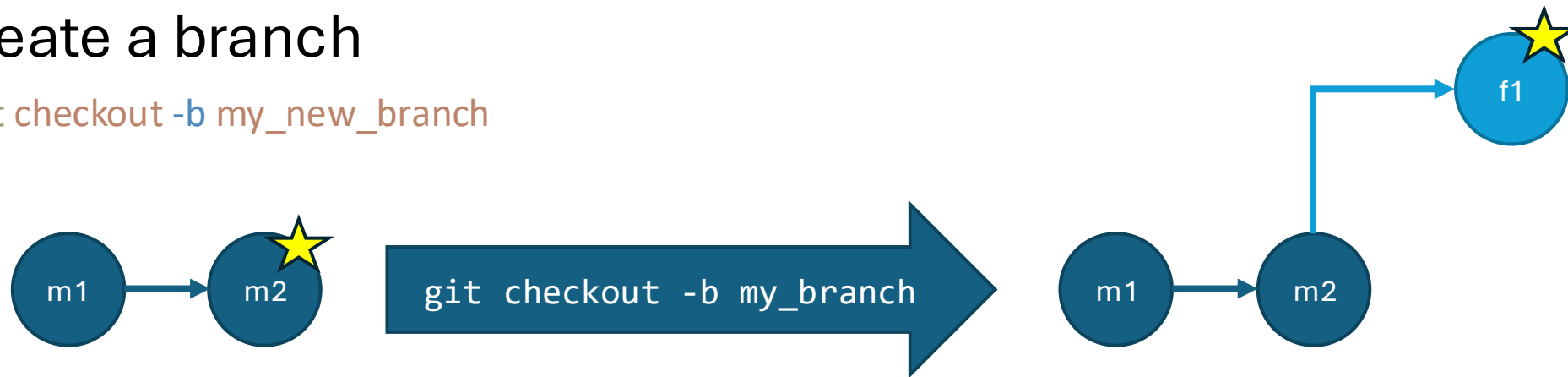
Creating Branches

- Good practice to first pull any recent changes to the repository

```
git checkout main  
git pull
```

- Create a branch

```
git checkout -b my_new_branch
```



Creating Branches

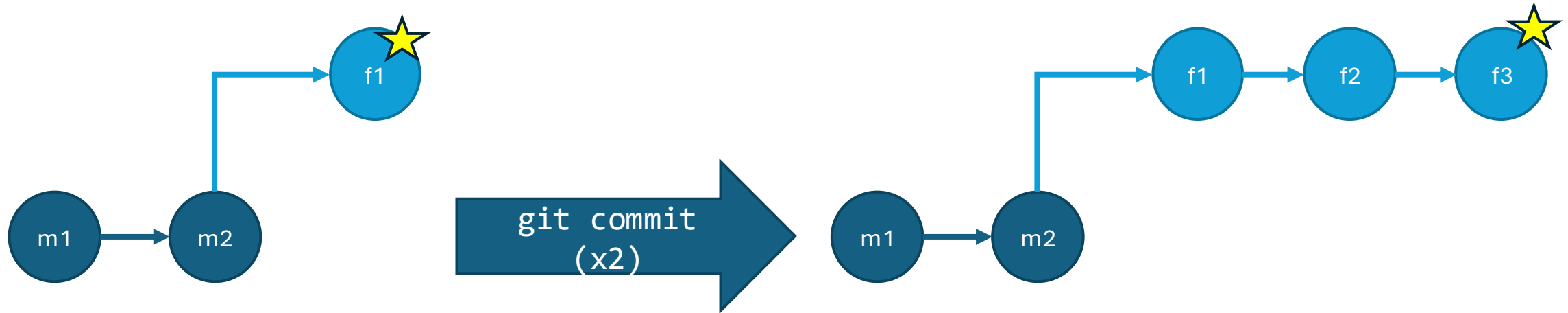
- Good practice to first pull any recent changes to the repository

```
git checkout main  
git pull
```

- Create a branch

```
git checkout -b my_new_branch
```

- New commits will be added to this branch



Creating Branches

- Good practice to first pull any recent changes to the repository

```
git checkout main  
git pull
```

- Create a branch- use short, descriptive names

```
git checkout -b my_new_branch
```

- New commits will be added to this branch
- Publish your branch

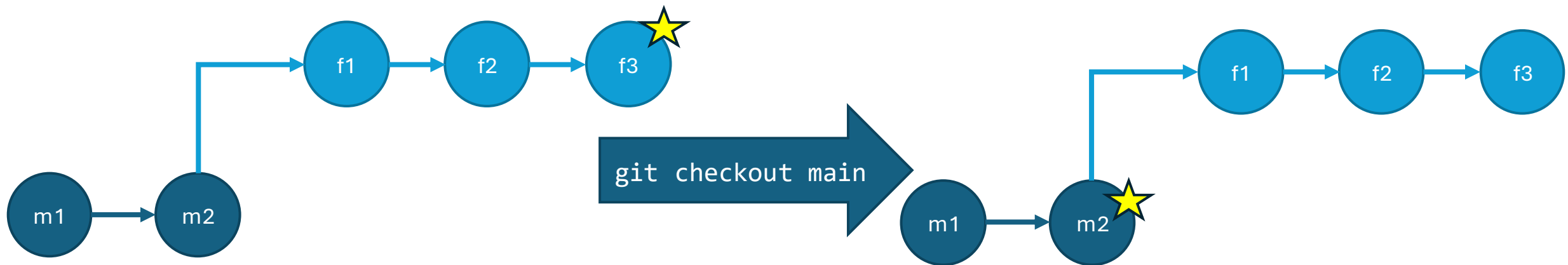
```
git push --set-upstream origin my_new_branch
```

- Add new commits to the remote repository

```
git push
```

Changing Branches

- Changing branches using the command `git checkout`
- For example, to change to the main branch, run
`git checkout main`
- The `git status` command will display which branch you are on



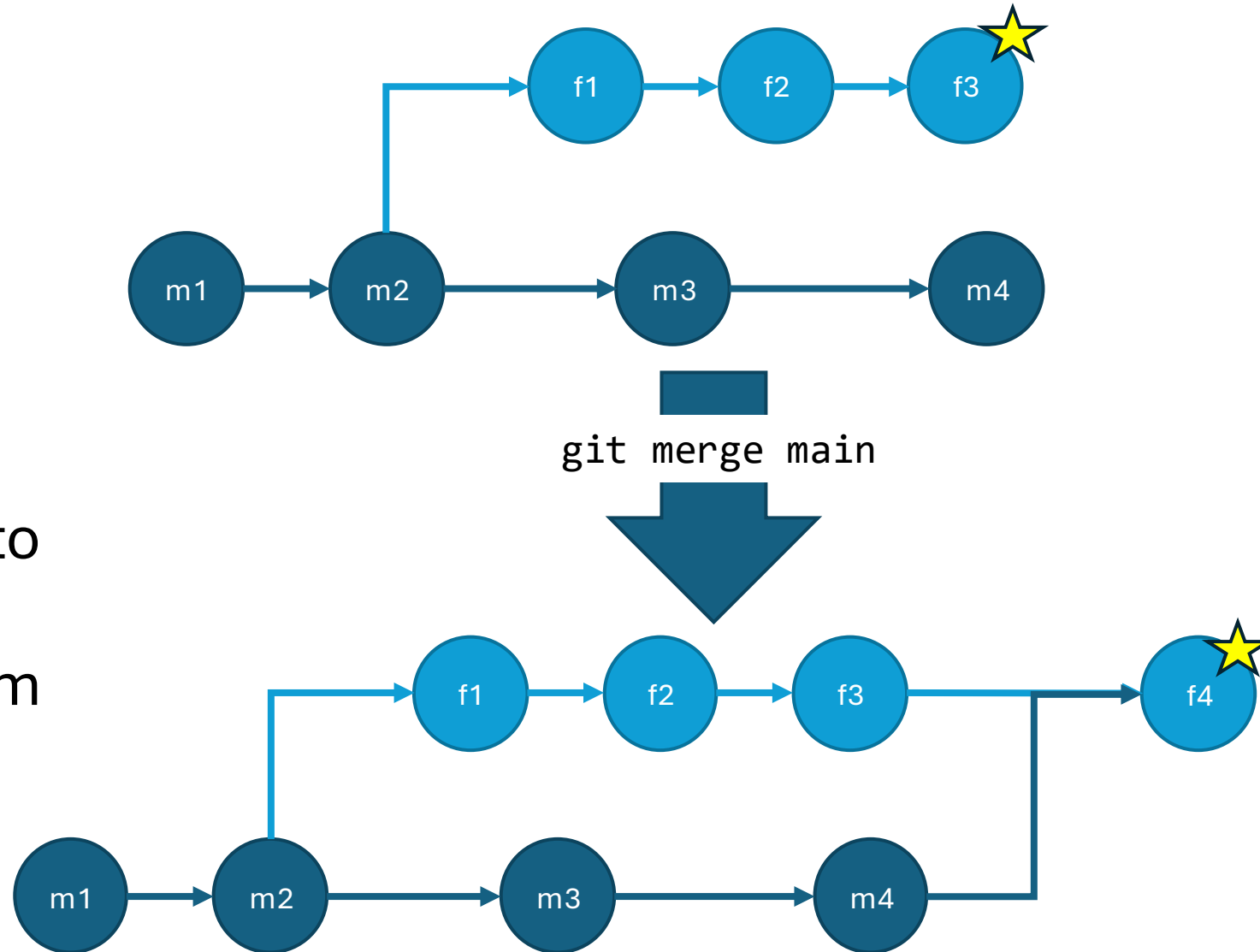
Merging Branches

- Bring changes from one branch into another with a merge

`git merge branch_name`

- This command brings changes from `branch_name` to the current branch
- To merge new changes from main into my_new_branch

`git checkout my_new_branch`
`git merge main`



Merging Branches

- Bring commits from one branch into another with a merge

```
git merge branch_name
```

- This command brings commits from **branch_name** to the current branch
- To merge new changes from main into a my_new_branch

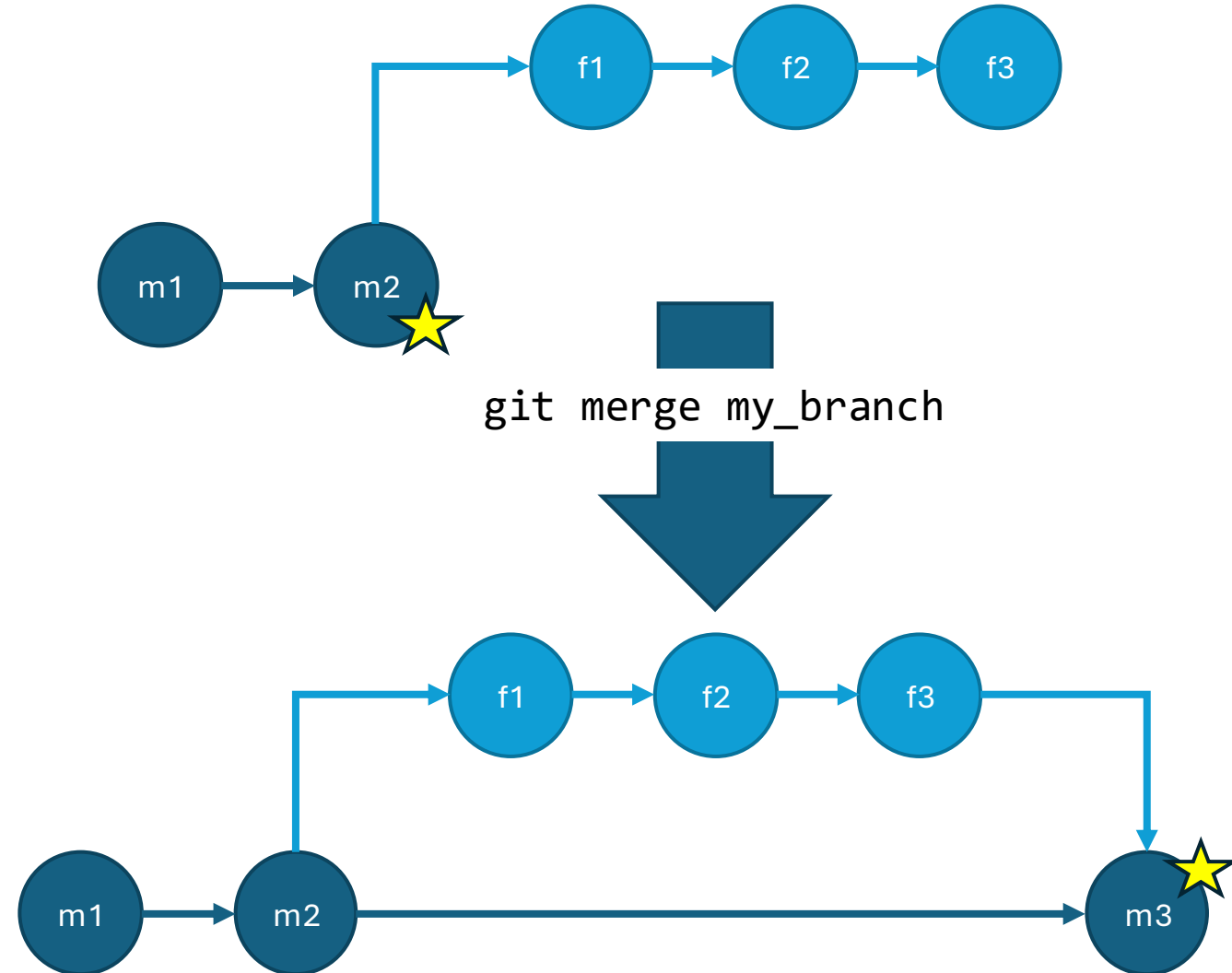
```
git checkout my_new_branch
```

```
git merge main
```

- When development is complete on my_new_branch it can be merged into main with

```
git checkout main
```

```
git merge my_new_branch
```



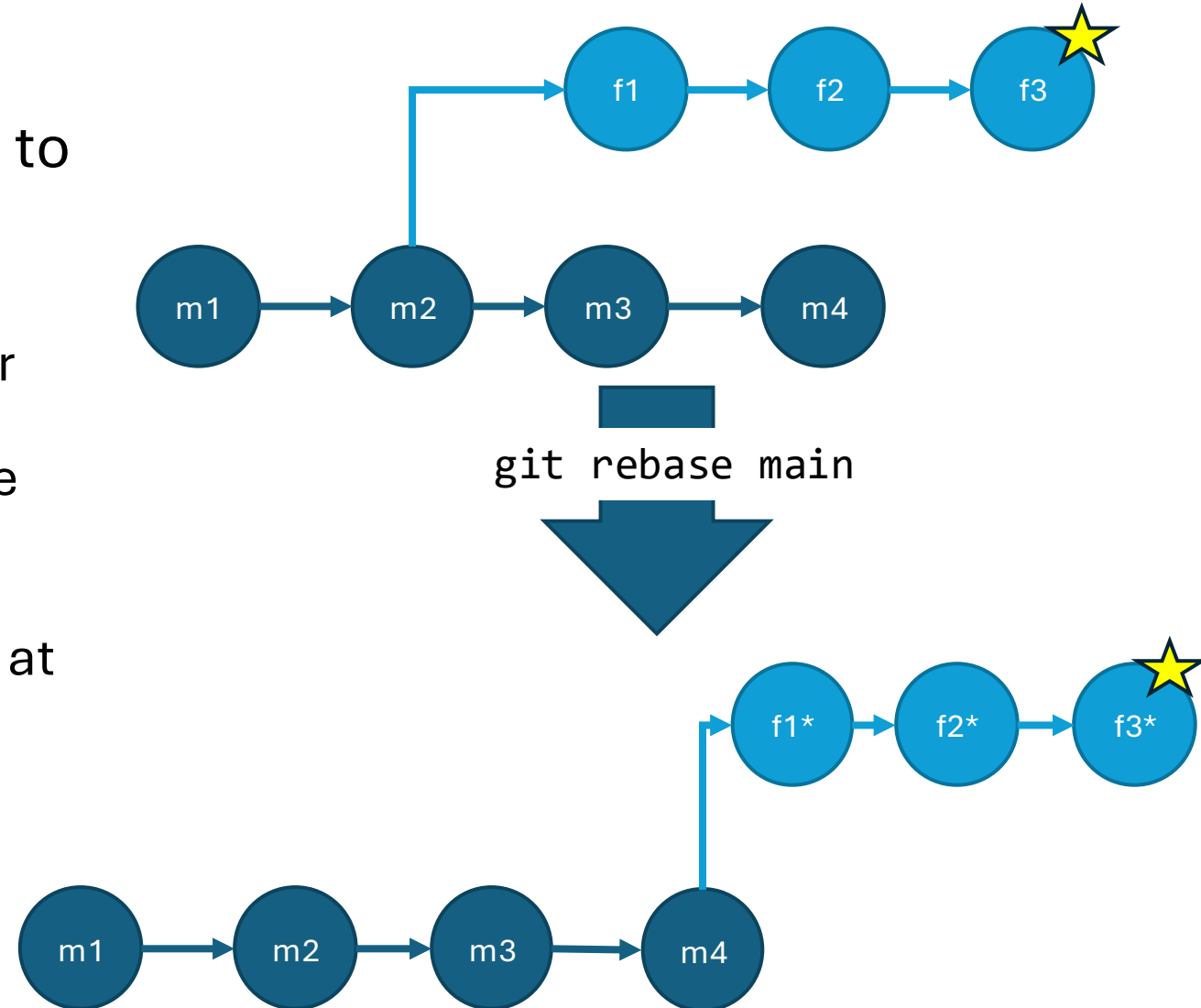
Merge Conflicts

- Git will do its best to merge automatically
- Merge conflicts occur when when git can't do this
 - Usually when the same line is changed in both branches
- Merge conflicts must be resolved before completing the merge
 - Manually using a text editor
 - Using a visual mergetool
 - Communicate with collaborators if necessary
- Ways to avoid
 - Modular code- work on one feature less likely to impact another
 - Regularly pull any changes from main and merge into feature branch
 - When development of a feature is complete, check for and merge any changes from main into the feature branch before merging the feature branch into main
- Resources
 - <https://coderefinery.github.io/git-intro/conflicts/>
 - <https://www.atlassian.com/git/tutorials/using-branches/merge-conflicts>



Merge vs Rebase

- Rebase sometimes an alternative to merging
- For example
 - Two new main branch commits after branch is created
 - Rebase your feature branch onto the main branch with
`git rebase main`
 - This “replays” your branch commits at the end of the main branch
- Rewrites history
- Should never be done on a public branch or commit



Updating the Basic Git Pattern

1. Prepare to make changes

`git status`

`git pull`

2. Create a new branch

`git checkout -b my-branch`

3. Push your branch to the remote repository

`git push --set-upstream origin my-branch`

4. Make changes

5. Commit your changes

`git add filename`

`git commit -m "commit message"`

6. Push your changes to the remote repository

`git push`

7. Repeat steps 1-5 as needed

8. Merge any changes from main into your branch

`git checkout main`

`git pull`

`git checkout my-branch`

`git merge main`

9. Merge your branch

Activity (15 mins)

- Everyone
 - Create a new branch in your local clone: `git checkout -b branch_name`
 - Publish the branch to the remote repository: `git push --set-upstream origin branch_name`
 - Make a small change, commit, and push
 - Look at the remote repository, look at your branch and your team's branches
- All but one team member
 - Merge the branch

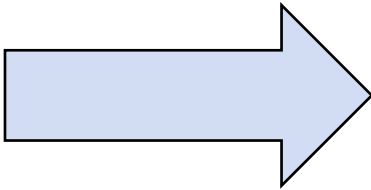
```
git pull
git checkout main
git merge branch_name
git push
```

- Did anyone run into merge conflicts? How might this have been prevented?
- There should be one unmerged branch at the end of this activity



Outline

- Introduction
- Branches
- Pull Requests
- Git Workflows



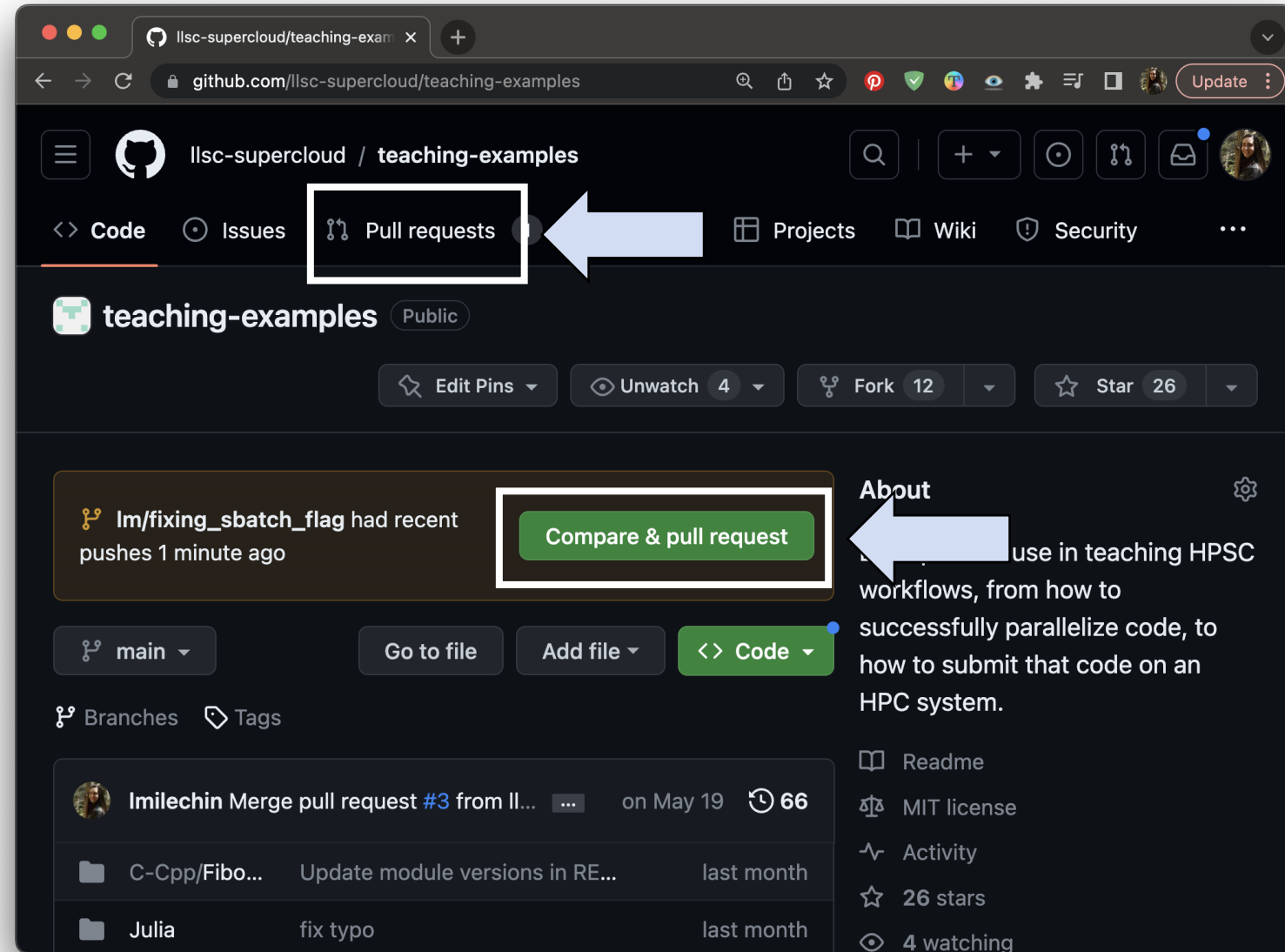
Pull Requests

- A process on GitHub to request a branch be merged into another branch
 - You are asking for your branch to be “pulled” into another branch
- GitLab uses the term “Merge Request”
- Allows code review and testing before merge
 - Alerts team and others about changes in branch before merge
 - Discussions ensue with possible follow up commits
 - Can request reviewer (may want particular expertise, may want to give someone knowledge)
- Set policies for merge
 - Enforce rules such as coding standards
 - Minimum number of reviewers
 - Protected branches – who can merge to certain branches etc.



Creating a Pull Request

- First make sure you've pushed all commits with `git status` and `git push`
- Go to your branch in GitHub
- If you pushed changes recently you may see a button that says "Compare and Pull Request", click if it is there
- If not click "Pull Requests"
 - If you click "Pull Requests" select "New Pull Request" and change the "compare" dropdown to your branch name





Creating a Pull Request

- Fill out the form with
 - A title
 - A description of the changes you made and why you made them, include any other relevant information
- When you are done click “Create pull request”

The screenshot shows the GitHub interface for creating a pull request. At the top, it says 'Open a pull request' and 'Create a new pull request by comparing changes across two branches. If you need to, you can also c'. Below this, there are dropdowns for 'base: main' and 'compare: lm/fixing_sbatach_flag', followed by a green checkmark and the text 'Able to merge. These branches can'. The main form has a title field containing 'Fixing --ntasks-per-node flag in Distributed Pytorch Example' and a description field containing 'Updating incorrect sbatch flag in Distributed Pytorch job submission script. The flag is updated from "--tasks-per-node" to "--ntasks-per-node".'. At the bottom right, there is a green button labeled 'Create pull request' with a dropdown arrow. A large blue arrow points to this button.

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also c

base: main compare: lm/fixing_sbatach_flag ✓ Able to merge. These branches can

Fixing --ntasks-per-node flag in Distributed Pytorch Example

Write Preview

H B I ≡ <> 🔗 ≡ ≡ ≡ @ ↗ ↶ ☑

Updating incorrect sbatch flag in Distributed Pytorch job submission script. The flag is updated from "--tasks-per-node" to "--ntasks-per-node".

Attach files by dragging & dropping, selecting or pasting them. 📎

Create pull request



Anatomy of a Pull Request

- The PR number is displayed next to the PR title
- In the conversation tab you'll see a timeline of comments and commits
 - The first "comment" is the initial description
 - Scroll to the bottom of the to add more comments
- It will also show whether there are any merge conflicts
 - Resolve any conflicts locally and push the commit to update

The screenshot shows a GitHub Pull Request (PR) titled "Fixing --ntasks-per-node flag in Distributed Pytorch". The PR number is 4, indicated by a blue arrow labeled "The PR Number". The PR is open, showing a conversation tab with 0 comments, 1 commit, 0 checks, and 1 file changed. The commit message is "fixing --ntasks-per-node flag" with hash 397ee98. The PR description, indicated by a blue arrow labeled "Your Description", states: "Updating incorrect sbatch flag in Distributed Pytorch job submission script. The flag is updated from '--tasks-per-node' to '--ntasks-per-node'." The PR is assigned to no one. The PR status, indicated by a blue arrow labeled "Conflict Status", shows "This branch has no conflicts with the base branch" and "Merging can be performed automatically." The PR is created by imilechin, a member of the repository.

Fixing --ntasks-per-node flag in Distributed Pytorch
Example #4

Open imilechin wants to merge 1 commit into main from lm/fixing_sbatch_flag

Conversation 0 Commits 1 Checks 0 Files changed 1 +1 -1

imilechin commented 1 minute ago Member

Updating incorrect sbatch flag in Distributed Pytorch job submission script. The flag is updated from "--tasks-per-node" to "--ntasks-per-node".

fixing --ntasks-per-node flag 397ee98

Add more commits by pushing to the lm/fixing_sbatch_flag branch on llsc-supercloud/teaching-examples.

Continuous integration has not been set up
GitHub Actions and several other apps can be used to automatically catch bugs and enforce style.

This branch has no conflicts with the base branch
Merging can be performed automatically.

Reviewers
Assignees
Labels
Projects
Milestones
Development



Anatomy of a Pull Request

- The PR number is displayed next to the PR title
- In the conversation tab you'll see a timeline of comments and commits
 - The first "comment" is the initial description
 - Scroll to the bottom of the to add more comments
- It will also show whether there are any merge conflicts
 - Resolve any conflicts locally and push the commit to update
- Continuous Integration

The screenshot shows a GitHub Pull Request (PR) page for the repository 'llsc-supercloud/supercloud-docs-edit'. The PR title is 'fixing broken link to external openssh how-to #2' and it is from the 'lm/openssh_fix' branch to the 'main' branch. The PR is currently open and has one commit.

The main content area shows a warning message: 'Your protected branch rules for your main branch won't be enforced on this private repository until you upgrade your organization account to a GitHub Team or Enterprise account.' Below this is a button labeled 'Upgrade'.

Below the warning, there is a section for 'Some checks haven't completed yet' with a link to 'Hide all checks'. It shows '1 in progress check'.

Below the checks, there is a section for 'This branch has no conflicts with the base branch' with a link to 'Details'. It states 'Merging can be performed automatically.'

At the bottom, there is a button labeled 'Merge pull request' and a dropdown menu. Below the button, it says 'You can also open this in GitHub Desktop or view command line instructions.'

On the right side, there is a sidebar with sections: 'Projects' (None yet), 'Milestone' (No milestone), 'Development' (Successfully merging this pull request may close these issues. None yet), and '0 participants'. There is also a 'Lock conversation' button.

A blue arrow points to the 'Details' link in the 'Some checks haven't completed yet' section, with a text box that says 'GitHub Workflow to create Preview'.



Anatomy of a Pull Request

The screenshot shows the GitHub interface for a pull request titled "Fixing --ntasks-per-node flag in Distributed Pytorch Example #4". The "Commits" tab is selected, showing a single commit by "Imilechin" with the message "fixing --ntasks-per-node flag". The commit hash is 397ee98. The interface includes navigation tabs for Code, Issues, Pull requests (2), Actions, Projects, Wiki, Security, and Insights. The commit details section shows the commit message and the user's profile picture.

The Commits tab shows all the commits in the PR

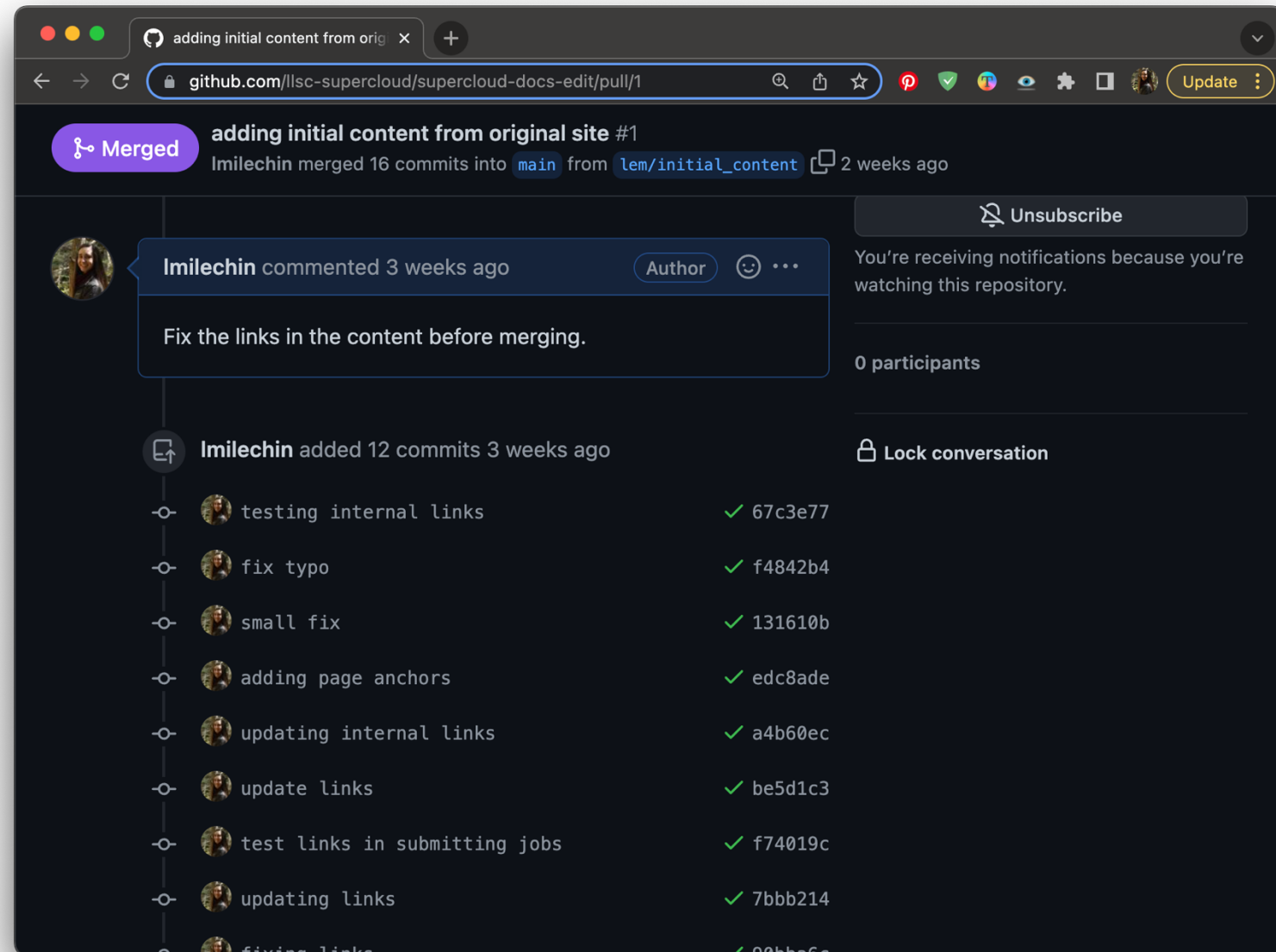
The screenshot shows the same GitHub pull request page, but with the "Files changed" tab selected. It displays a diff for the file "Python/pytorch/Distributed/batch.sh". The diff shows a change from line 5: `#SBATCH --tasks-per-node=2` to `#SBATCH --ntasks-per-node=2`. The interface includes navigation tabs for Code, Issues, Pull requests (2), Actions, Projects, Wiki, Security, and Insights. The "Files changed" tab shows a list of files and a "Review changes" button.

Click “Files Changed” to review changes



Updating a Pull Request

- Others on the team should review the changes
- Feedback can be given using the comment field at the bottom of the Conversation Tab
- To update the PR, make updates in your branch and push the commits
- New commits will be added to the PR automatically





Accepting and Merging a Pull Request

- Once the PR has been reviewed it is ready to merge
- Click “Merge Pull Request” to merge, then click “Confirm merge”
- After merging, you can click “delete branch” if you’d like, new material from here should be in a new branch
- When the PR is merged the web page will be updated

The screenshot shows a GitHub pull request merge confirmation interface. At the top, a green button labeled "Open" with a key icon is next to the title "Fixing --ntasks-per-node flag in Distributed Pytorch Example #4". Below the title, it says "lmilechin wants to merge 1 commit into `main` from `lm/fixing_sbatch_flag`". A description box contains the text: "Updating incorrect sbatch flag in Distributed Pytorch job submission script. The flag is updated from '--tasks-per-node' to '--ntasks-per-node'." Below this is a commit preview for "fixing --ntasks-per-node flag" with commit hash "397ee98". A message says "Add more commits by pushing to the `lm/fixing_sbatch_flag` branch on `llsc-supercloud/teaching-examples`." At the bottom, a merge summary box shows "Merge pull request #4 from llsc-supercloud/lm/fixing_sbatch_flag" and "Fixing --ntasks-per-node flag in Distributed Pytorch Example". Below the summary, it states "This commit will be authored by 13963460+lmilechin@users.noreply.github.com". At the very bottom are two buttons: "Confirm merge" (green) and "Cancel" (grey).



Accepting and Merging a Pull Request

- Once the PR has been reviewed it is ready to merge
- Click “Merge Pull Request” to merge, then click “Confirm merge”
- After merging, you can click “delete branch” if you’d like, new material from here should be in a new branch
- When the PR is merged the web page will be updated

The screenshot displays a GitHub pull request titled "Fixing --ntasks-per-node flag in Distributed Pytorch Example #4". The status is "Merged", and it shows that "lmilechin merged 1 commit into main from lm/fixing_sbatch_flag". A description of the changes states: "Updating incorrect sbatch flag in Distributed Pytorch job submission script. The flag is updated from '--tasks-per-node' to '--ntasks-per-node'." Below this, a commit history shows the merge of commit 2e5d1d4 into the main branch. A success message indicates the pull request was successfully merged and closed, with a "Delete branch" button. At the bottom, there is a "Write" and "Preview" section with a rich text editor toolbar.



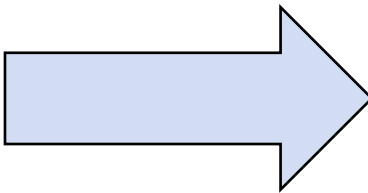
Activity (15 mins)

- Your team's repository should have one unmerged branch, if not, create one now and publish the branch
- Everyone: Log into GitHub
- Branch owner: create a Pull Request
- Team: Review the Pull Request
 - Look at the commits and files changed
 - Do the changes look okay? Comment if not
 - Are there any merge conflicts?
- Branch owner: address any merge conflicts or comments
- Team: comment when it looks good to merge
- Once approved merge the pull request
- In both this activity and the last one your branch was eventually merged into the main branch. How does this experience compare to the previous activity?
- If you have extra time, the remaining team members can create their own Pull Requests with new branches.



Outline

- Introduction
- Branches
- Pull Requests
- Git Workflows



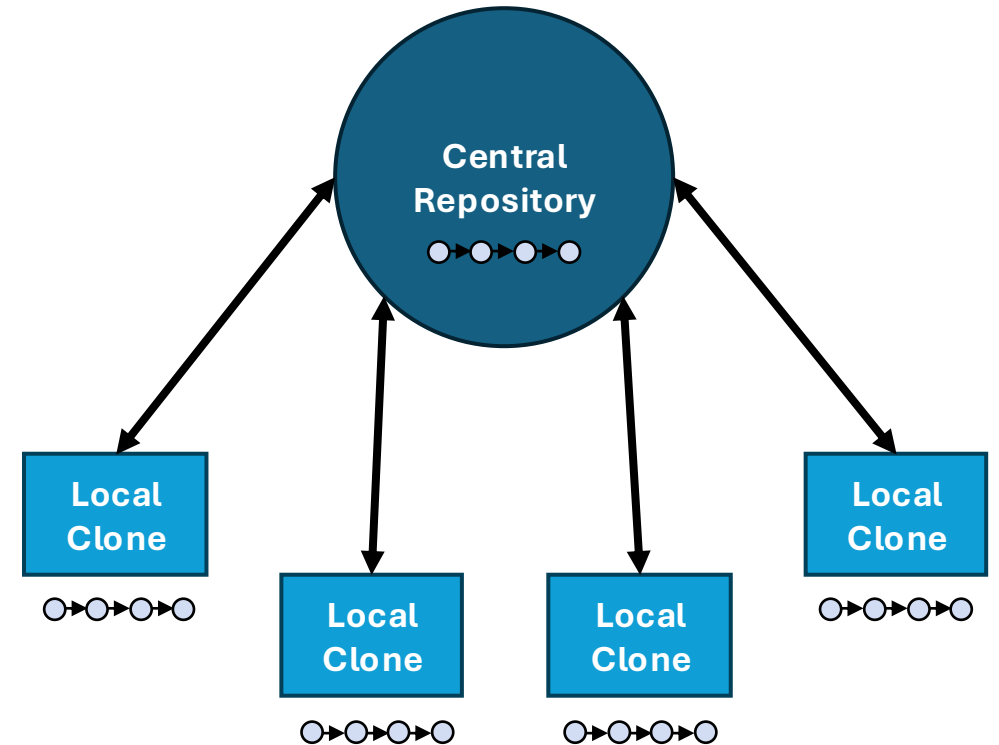


Git Workflows

- A process for how your team is going to work with a repository, including
 - How you are going to use branches (branching strategy)
 - Feature branches, "lifetime" branches (production, development, release branches)
 - What is considered the "latest version", or what is "production" vs "in development"
 - How branches are merged
 - When you are making pull requests and into what
 - Testing
 - Whether you are working in a single, central repository, or separate forked repositories
- Help everyone stay on the same page about the state of the code
- Enable communication
- Tell us
 - What branches are protected
 - What is tested and when
 - What branches are stable

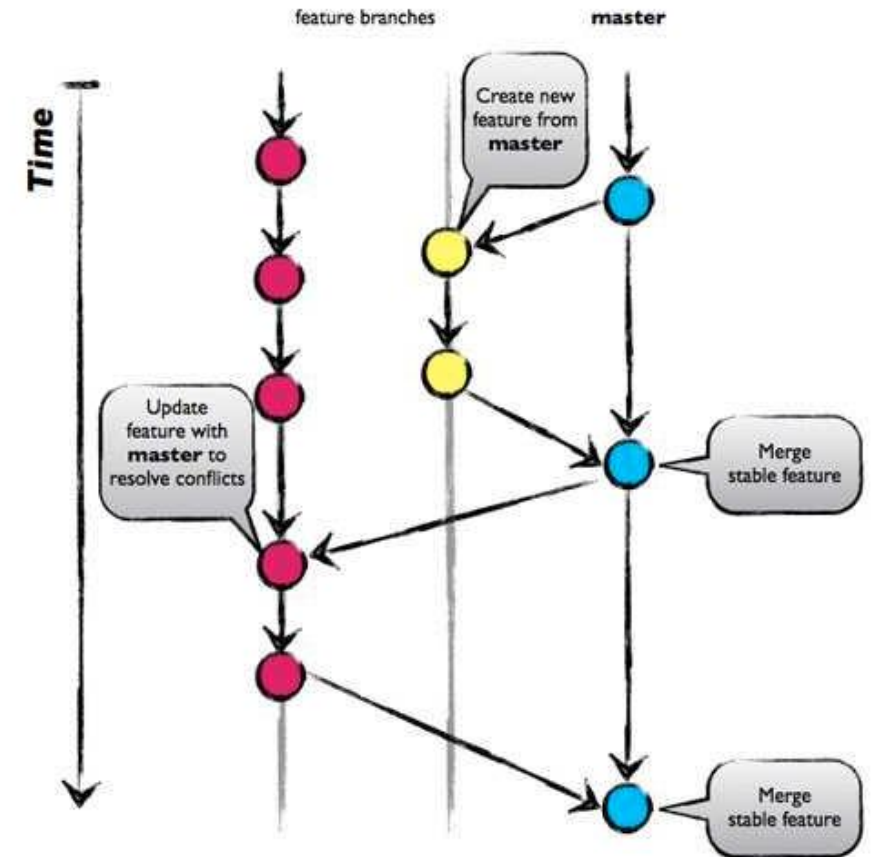
Centralized or Trunk-Based Workflow

- Central repository with one main branch
- Developers work in their local clone and push less frequently
- Central repository meant to be stable
- Resolve conflicts locally before pushing
- Not the best for collaborative development



GitHub Flow

- A feature branching workflow
- Published as viable alternative to Git Flow (we'll see this next)
- No structured release schedule
- Continuous deployment & continuous integration
- Simpler workflow
- Key Ideas
 - All commits in the main branch are deployable
 - Base feature branches off main
 - Push local repository to remote constantly
 - Open Pull Requests early to start dialogue
 - Merge into main after Pull Request review



<https://scottchacon.com/2011/08/31/github-flow>

David M. Rogers, Collaborative Software Development in Better Scientific Software tutorial, in ISC High Performance, Hamburg, Germany and online, 2023. DOI: [10.6084/m9.figshare.22790762](https://doi.org/10.6084/m9.figshare.22790762).

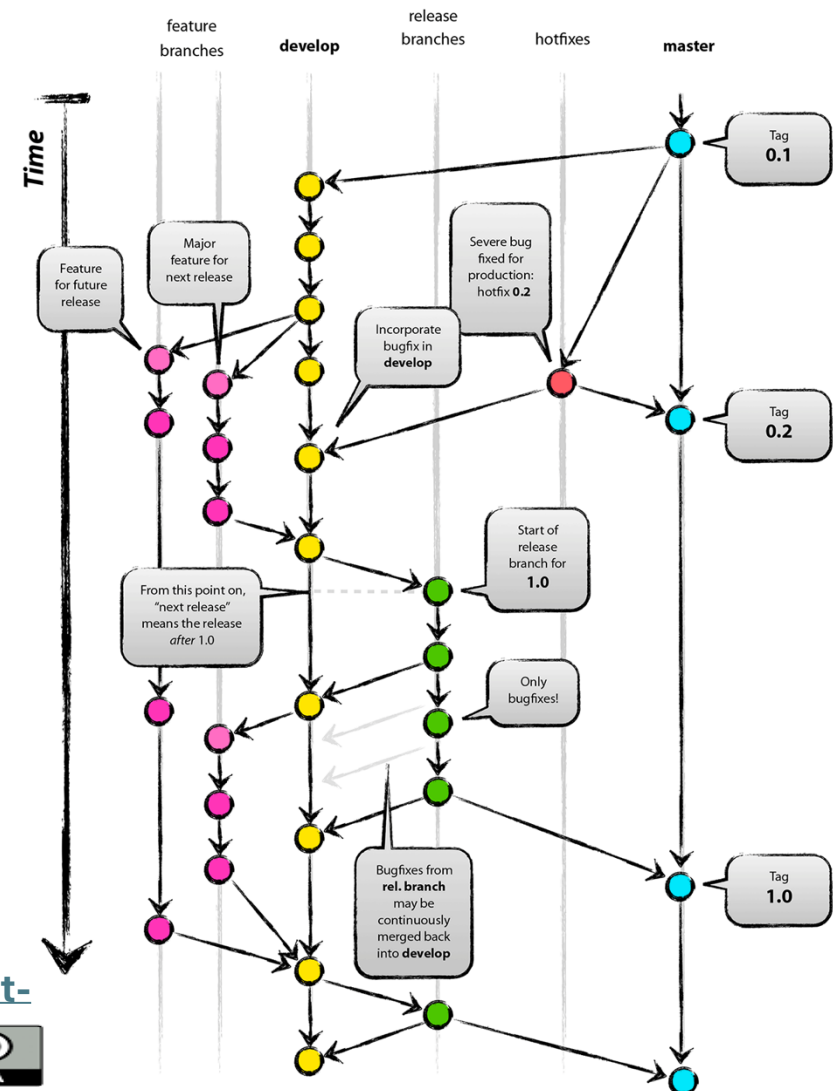
Git Flow

- Full-featured workflow
- Increased complexity
- Designed for Software with official releases
- Feature branches based off of develop
- [Git extensions](#) to enforce policy
- How are develop and main synchronized?
- Where do merge conflicts occur and how are they resolved?

Author: Vincent Driessen

Original Blog: <https://nvie.com/posts/a-successful-git-branching-model/>

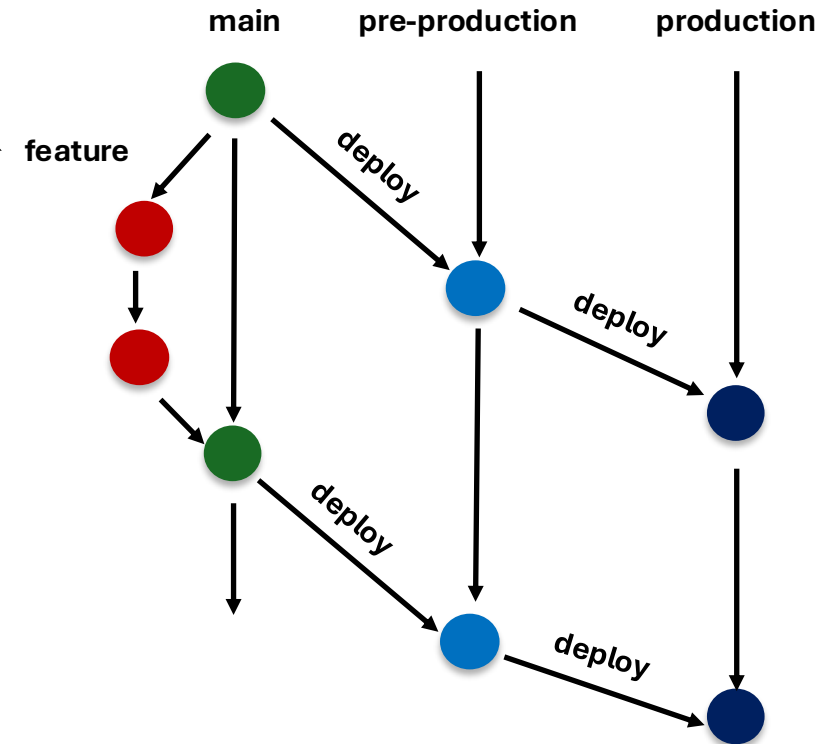
License: Creative Commons



David M. Rogers, Collaborative Software Development in Better Scientific Software tutorial, in ISC High Performance, Hamburg, Germany and online, 2023. DOI: [10.6084/m9.figshare.22790762](https://doi.org/10.6084/m9.figshare.22790762).

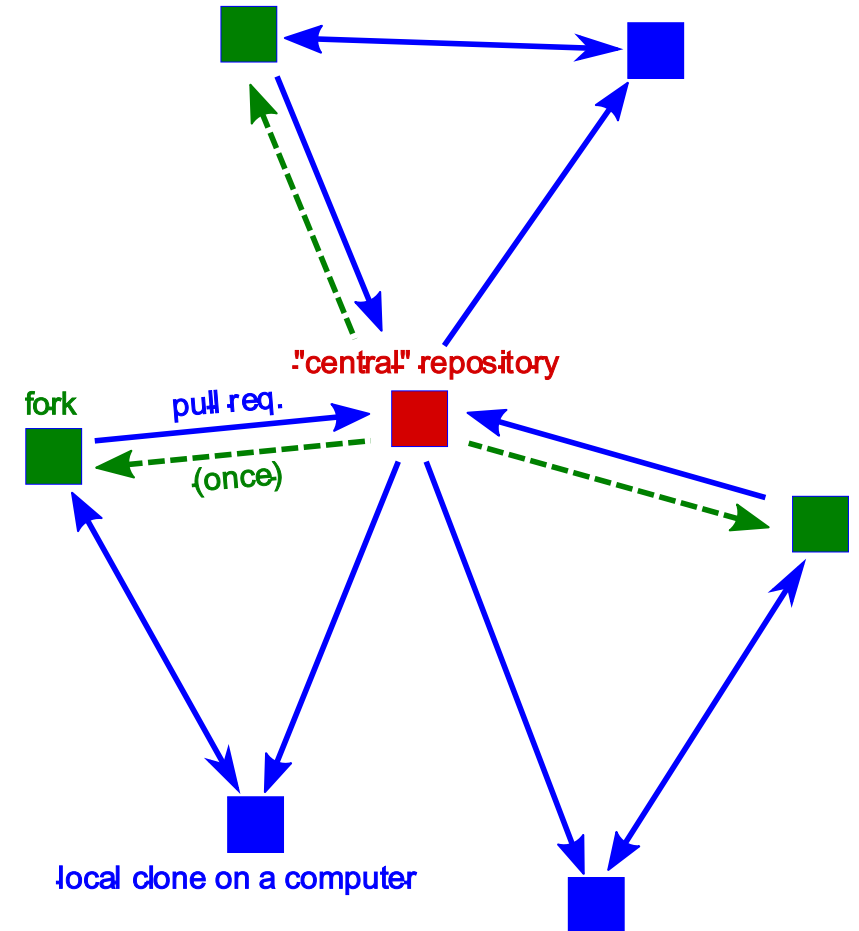
GitLab Flow

- Published as viable alternative to Git Flow & GitHub Flow
- Semi-structured release schedule
- Simplifies difficulties in synchronizing infinite lifetime branches
- Key Ideas
 - Main branch is staging area
 - Mature code in main flows downstream into pre-production & production infinite lifetime branches
 - Allow for release branches with downstream flow
 - Fixes made upstream & merged into main
 - Fixes cherry picked into release branch



Forks in Workflows

- So far we have assumed branches are created in the main repository- not feasible for some projects
- Contributors:
 - Fork the repository and mainly interact with their fork
 - Create branches in fork and create a pull request from the fork to the original repository to merge changes
- Anybody can propose contributions without asking for advance permission (to public projects)
- Maintainer still has full control over what is merged
- Challenge: contributors now have more than one remote to work with
- This is used by almost all large (and small) open-source projects these days





Tips for Using Forks

- Set and name remotes to help keep track of original and forked repositories

```
git remote add upstream https://github.com/project_team/project.git  
git remote add my-fork https://github.com/me/project.git
```

- View remotes with

```
git remote -v
```

- Retrieve upstream changes with

```
git fetch upstream
```

- Merge upstream changes into your fork's main branch

```
git checkout main # this is the main branch for your fork  
git merge upstream/main
```

- Retrieve any upstream changes and merge into your main and feature branch frequently, especially before you create a Pull Request

Guidelines for establishing a Git Workflow

- Communicating it to your collaborators
 - Include workflow in Contributing Guide
 - Establish conventions for branch naming (issues, major/minor versions)
 - Refer to the Contributing Guide in your agent configuration file (AGENTS.md, CLAUDE.md, or copilot-instructions.md)
- Enforce workflow
 - [Branch protections](#)
 - Limiting who can push/merge
 - Testing & review requirements

Guidelines for establishing a Git Workflow

- Adopt what is good for your team
 - Consider team culture and project challenges
 - Assess what is and isn't feasible/acceptable
 - Start with simplest and add complexity where and when necessary
- Be careful in shared spaces
 - Each person should work within their own clone
 - Separate “production” clone for running

Activity (10 mins)

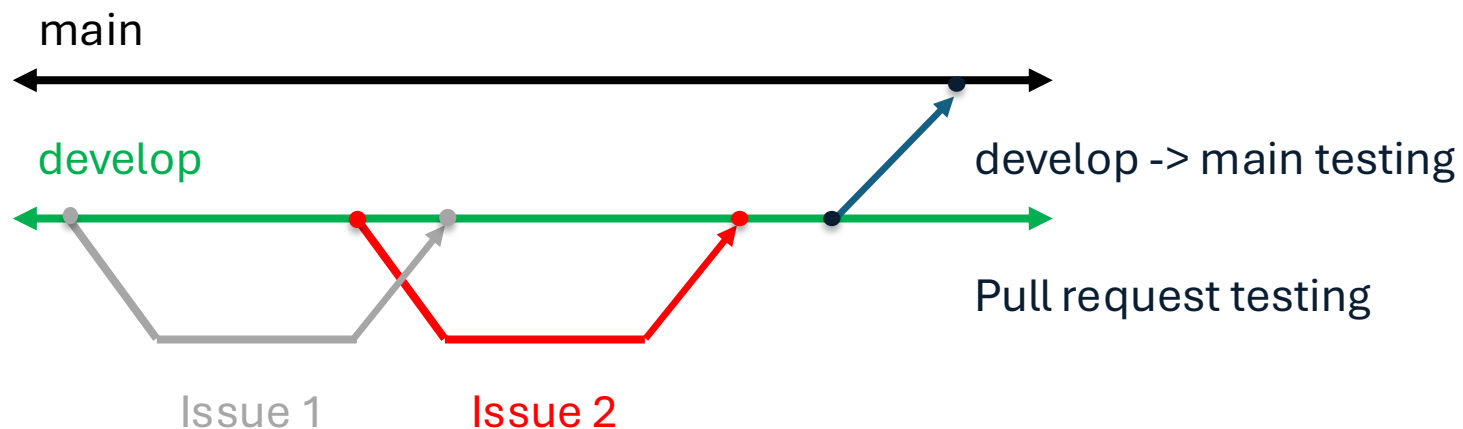
- Discuss amongst your group which workflow will work best for your group
- Set up your repository accordingly
- Start a contributor guide that explains the workflow and any rules for contributing
- If time allows, discuss any projects that your group members work on collaboratively with others. Is there a specified workflow for this project? How do you contribute to the project?



Backup

Current Trilinos Workflow

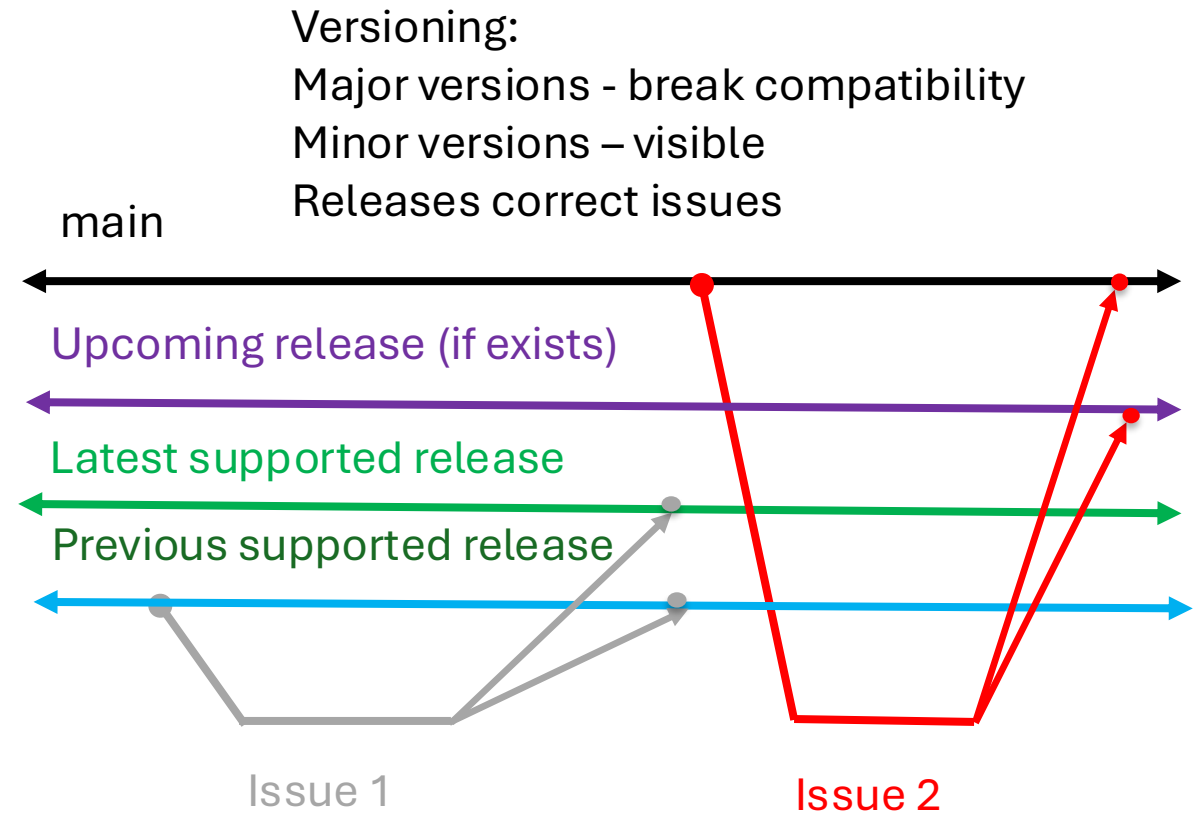
- Test-driven workflow
 - Feature branches start and end with develop
 - All changes to develop must come from GitHub pull requests
 - Feature branches are merged into develop only after passing pull request test suite
 - Change sets from develop are tested daily for integration into main
- Workflow designed so that
 - All commits in main are in develop
 - Merge conflicts exposed when integrating into develop
 - Merge conflicts never occur when promoting to main



<https://trilinos.github.io/>

Current Open MPI Workflow

- Workflow designed so that
 - Support two most recent releases
 - Issues are addressed on all applicable branches
 - All PR's reviewed by at least one core developer
 - Main and supported branches work at all times
 - Developers work on main or feature branches depending on complexity of the changes
- Testing
 - CI testing on PR's for any branch using Jenkins (limited set of compilers, hardware, tests)
 - Nightly testing on all branches using community-built MTT framework (more complex set of compilers, hardware, tests)
 - Additional testing for release candidates



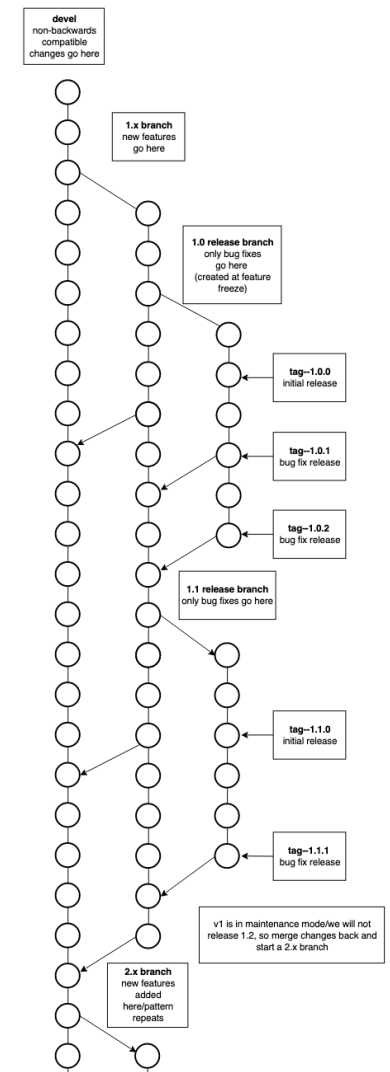
<https://www.openmpi.org>



<https://flecsi.github.io/flecsi>

Current FleCSI Workflow

- Workflow designed so that
 - All supported branches work at all times
 - Merge Requests are tested and reviewed
- Testing
 - Customized unit-testing framework based on Google Test
 - Special *gitlab-ci* branch - images and configuration files
- Versioning:
 - Incompatible - devel branch breaks compatibility with previous versions
 - Feature (1, 2 ...) named for major version
 - Release - (1.x, 2.x ...) named for major.minor version, correct issues, tags used for bug fixes.



David M. Rogers, Collaborative Software Development in Better Scientific Software tutorial, in ISC High Performance, Hamburg, Germany and online, 2023. DOI: [10.6084/m9.figshare.22790762](https://doi.org/10.6084/m9.figshare.22790762).